

ITE 2132 - Object Oriented Programming

Class And Object

1. Class:

- A class in Java is a blueprint or template for creating objects. It defines the properties (attributes) and behaviors (methods) that objects of that class will have.
- It acts as a blueprint because you can create multiple objects (instances) based on the same class. Each object created from a class will have its own set of data (attributes) and can perform actions (methods) independently of other objects of the same class.
- Classes are typically declared using the `class` keyword followed by the class name.

Example:

```
public class Car {  
    // Attributes  
    String make;  
    String model;  
    int year;  
  
    // Methods  
    public void start() {  
        // Code to start the car  
    }  
  
    public void drive() {  
        // Code to drive the car  
    }  
}
```

```
}  
}
```

2. Object:

- An object is an instance of a class. It's a concrete entity that exists in memory and represents a specific thing.
- When you create an object, you are creating a realization of the class blueprint. Each object has its own set of attributes (also known as instance variables or member variables) and can invoke methods defined in its class.
- Objects are created using the `new` keyword followed by the class name, and you can assign it to a reference variable.

Example:

```
public class Main {  
    public static void main(String[] args) {  
        // Creating objects of class Car  
        Car myCar = new Car();  
        Car anotherCar = new Car();  
  
        // Accessing attributes and invoking methods  
        myCar.make = "Toyota";  
        myCar.model = "Camry";  
        myCar.year = 2020;  
        myCar.start();  
        myCar.drive();  
  
        anotherCar.make = "Honda";  
        anotherCar.model = "Accord";  
        anotherCar.year = 2018;  
        anotherCar.start();  
        anotherCar.drive();  
    }  
}
```

```
}  
}
```

In this example, `myCar` and `anotherCar` are objects of the `Car` class. They each have their own set of attributes (`make` , `model` , `year`) and can invoke methods (`start()` , `drive()`) defined in the `Car` class. This is the essence of class and object theory in Java OOP.

Public vs Private

In Java, `public` and `private` are access modifiers used to control the visibility of classes, methods, and fields within a program. Here's a breakdown of each:

1. Public:

- When a class, method, or field is declared as `public` , it can be accessed from any other class in the same package or from any other package.
- This means that public elements are accessible outside of their own class and are part of the class's public interface.
- It's commonly used for methods and fields that need to be accessible from other parts of the program.

Example:

```
public class MyClass {  
    public int publicField;  
  
    public void publicMethod() {  
        // Code  
    }  
}
```

1. Private:

- When a class member is declared as `private` , it can only be accessed within the same class.

- Private members are not visible to classes outside of their own class, including subclasses.
- It's often used for fields and methods that should not be accessible or modified from outside the class.

Example:

```
public class MyClass {  
    private int privateField;  
  
    private void privateMethod() {  
        // Code  
    }  
}
```

Here's a summary of the key differences:

- `public`: Visible and accessible from any other class.
- `private`: Accessible only within the same class.

In Java, encapsulation is encouraged, so it's common to see instance variables declared as `private` and accessed and modified through public getter and setter methods. This helps maintain data integrity and provides controlled access to the class's state.

The difference between instance and local variables

In Java, instance variables and local variables are both types of variables, but they have different scopes and lifetimes within a program.

1. Instance Variables:

- Instance variables are declared within a class but outside of any method, constructor, or block.
- They are associated with objects of the class and are created when an object is instantiated (i.e., when `new` keyword is used).

- Each instance of the class (object) has its own copy of instance variables, which are initialized to default values if not explicitly initialized.
- Instance variables exist as long as the object they belong to exists. They are destroyed when the object is garbage collected.

Example:

```
public class MyClass {
    // Instance variables
    private int x; // Initialized to 0 by default
    private String name; // Initialized to null by default

    // Constructor
    public MyClass(int x, String name) {
        this.x = x;
        this.name = name;
    }
}
```

1. Local Variables:

- Local variables are declared within a method, constructor, or block and are accessible only within that specific scope.
- They are created when the method, constructor, or block is entered, and they are destroyed when it exits.
- Local variables must be explicitly initialized before they are used.

Example:

```
public class MyClass {
    // Method
    public void myMethod() {
        // Local variable
        int y = 10;
        String message = "Hello";
    }
}
```

```
        // Code using local variables
        System.out.println(message + ", y = " + y);
    }
}
```

Key differences between instance and local variables:

- **Scope:** Instance variables are accessible throughout the class (within all methods, constructors, and blocks), while local variables are accessible only within the method, constructor, or block where they are declared.
- **Lifetime:** Instance variables exist as long as the object they belong to exists, while local variables exist only within the scope in which they are declared and are destroyed when that scope exits.
- **Initialization:** Instance variables can have default values if not explicitly initialized, while local variables must be explicitly initialized before use.

Return Statement

In Java, the `return` statement is used to exit a method and optionally return a value to the caller of the method. Here are the key points about the `return` statement:

1. Syntax:

- The `return` statement is followed by an optional expression that specifies the value to be returned to the caller. If the method has a return type other than `void`, the `return` statement must provide a return value of that type.

```
return expression;
```

2. Exiting the Method:

- When the `return` statement is encountered in a method, it immediately exits the method and control returns to the caller.
- If the method has any remaining code after the `return` statement, it will not be executed.

3. Returning a Value:

- If the method has a return type other than `void`, it must return a value using the `return` statement.
- The return value must be compatible with the method's return type.
- If the method has a `void` return type, it does not return any value, and the `return` statement can be used without specifying an expression.

4. Multiple Return Statements:

- A method can have multiple `return` statements, but only one of them will be executed.
- The execution of the method stops as soon as a `return` statement is encountered.

5. Usage Examples:

- Returning a value:

```
public int add(int a, int b) {  
    return a + b;  
}
```

- Returning early from a method:

```
public void printPositive(int num) {  
    if (num <= 0) {  
        System.out.println("Number is not positive.");  
        return; // Exit the method early  
    }  
    System.out.println("Number is positive: " + num);  
}
```

- Using conditional expressions with `return`:

```
public int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
}
```

The `return` statement is fundamental for controlling the flow of execution within methods and for providing results back to the caller of the method.

Encapsulation

Encapsulation is one of the four fundamental principles of object-oriented programming (OOP) and is a key concept in Java. It refers to the bundling of data (attributes) and methods that operate on the data into a single unit, called a class. The class then controls access to its data by hiding its implementation details and providing a well-defined interface for interacting with the data.

Here are the key aspects of encapsulation in Java:

1. Access Modifiers:

- Java provides access modifiers (`public`, `private`, `protected`, and package-private) to control the visibility of classes, methods, and fields.
- Encapsulation often involves declaring fields as `private` to hide them from direct access outside the class.

2. Private Fields:

- Encapsulated classes typically have their fields declared as `private`.
- This prevents direct access to the fields from outside the class, ensuring data integrity and encapsulation.

Example:

```
public class Car {  
    private String make;  
    private String model;  
    private int year;  
  
    // Constructor, methods, etc.  
}
```

1. Getter and Setter Methods:

- Encapsulated classes provide getter and setter methods to access and modify the private fields.
- Getter methods allow access to the values of private fields.
- Setter methods allow modification of the values of private fields while enforcing any necessary validation logic.

Example:

```
public class Car {
    private String make;
    private String model;
    private int year;

    // Getter methods
    public String getMake() {
        return make;
    }

    public String getModel() {
        return model;
    }

    public int getYear() {
        return year;
    }

    // Setter methods
    public void setMake(String make) {
        this.make = make;
    }

    public void setModel(String model) {
        this.model = model;
    }
}
```

```
public void setYear(int year) {  
    this.year = year;  
}  
}
```

1. Encapsulation Benefits:

- Protects the integrity of the data by preventing unauthorized access and modification.
- Hides the implementation details of a class, allowing for easier maintenance and evolution of the codebase.
- Facilitates the principle of abstraction, where complex systems can be represented by simpler interfaces.

By encapsulating data and providing controlled access through methods, Java promotes secure, maintainable, and flexible codebases. This approach helps in building robust software systems that are easier to understand, maintain, and extend.

Static Methods

Static methods in Java are methods that belong to a class rather than to instances (objects) of the class. These methods are associated with the class itself, not with any particular object instance. Here are the key points about static methods:

1. Declaration:

- Static methods are declared using the `static` keyword before the return type in the method signature.
- They can be called directly on the class, without needing to create an instance of the class.

Example of a static method:

```
public class MyClass {  
    public static void myStaticMethod() {  
        // Method code  
    }  
}
```

1. Accessing Static Methods:

- Static methods can be called using the class name, followed by the dot operator (.), and then the method name.
- They can also be called using an object reference, but this is generally discouraged because it can be confusing.

Example of calling a static method:

```
MyClass.myStaticMethod(); // Calling using class name
```

1. Accessing Instance Variables:

- Static methods cannot directly access instance variables (non-static fields) of the class unless they are explicitly given an instance of the class.
- They can only directly access static variables (class variables) and other static methods.

2. Common Use Cases:

- Utility methods: Static methods are often used to define utility methods that perform common tasks and do not rely on the state of any particular object.
- Factory methods: Static methods can be used as factory methods to create instances of the class.
- Helper methods: They are useful for defining helper methods that are used across multiple instances of the class.

Example of a utility method:

```
public class MathUtils {  
    public static int add(int a, int b) {  
        return a + b;  
    }  
}
```

Static methods are an essential part of Java programming, providing a way to define behavior associated with a class itself, rather than with instances of the class. However, it's important to use them judiciously and be aware of their limitations, especially regarding access to instance variables.

Method Overloading

Method overloading is a feature in Java that allows a class to have multiple methods with the same name but different parameter lists within the same class. Here are the key points about method overloading:

1. Method Signature:

- For method overloading, methods must have the same name but different parameter lists. The parameter lists can differ in terms of the number of parameters, types of parameters, or both.

2. Return Type:

- The return type of the method can be the same or different for overloaded methods. Method overloading is not based on the return type; it's based on the method signature (name and parameters).

3. Access Modifier:

- Overloaded methods can have different access modifiers (e.g., `public`, `private`, `protected`, or package-private) in the same class.

4. Example:

```
public class Calculator {  
    public int add(int a, int b) {
```

```

        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }
}

```

5. Calling Overloaded Methods:

- When a method is called, Java matches the method call with the appropriate method based on the number and type of arguments provided.
- If no exact match is found, Java attempts to perform automatic type conversion (e.g., widening conversions) to find a compatible method.

6. Automatic Type Promotion:

- In method overloading, automatic type promotion occurs when there is no exact match for the parameters provided in the method call.
- Java automatically promotes the arguments to the next data type, if possible, to match a compatible method.

Example of automatic type promotion:

```

public class MyClass {
    public void foo(int x) {
        System.out.println("int");
    }

    public void foo(double x) {
        System.out.println("double");
    }
}

```

```
MyClass obj = new MyClass();  
obj.foo(10); // Calls foo(int x)  
obj.foo(10.5); // Calls foo(double x)
```

Method overloading provides flexibility and readability to code by allowing methods with similar functionality to be grouped together under the same name. It's a way to achieve polymorphism at the compile-time.

Inheritance

Inheritance is one of the fundamental concepts of object-oriented programming (OOP), and Java fully supports it. It enables one class (subclass or derived class) to inherit properties and behaviors from another class (superclass or base class). Here are the key points about inheritance in Java:

1. Superclass and Subclass:

- Inheritance involves creating a relationship between two classes: a superclass and a subclass.
- The superclass is the class being inherited from, and the subclass is the class that inherits from the superclass.

2. Extends Keyword:

- In Java, inheritance is implemented using the `extends` keyword.
- To create a subclass, you use the `extends` keyword followed by the name of the superclass.

Example:

```
class Animal {  
    // Superclass (or base class)  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}
```

```
class Dog extends Animal {
    // Subclass (or derived class)
    void bark() {
        System.out.println("Dog is barking");
    }
}
```

1. Access to Superclass Members:

- A subclass inherits members (fields, methods) from its superclass.
- Subclasses can access the members of their superclass using the `super` keyword.

Example:

```
class Dog extends Animal {
    void bark() {
        super.eat(); // Call superclass method
        System.out.println("Dog is barking");
    }
}
```

1. Method Overriding:

- Subclasses can provide a specific implementation of a method that is already defined in their superclass. This is known as method overriding.
- The method signature (name and parameters) must be the same in both the superclass and the subclass.

2. Single Inheritance:

- Java supports single inheritance, meaning a class can only inherit from one superclass.
- However, a class can be a superclass for multiple subclasses, creating a hierarchical inheritance structure.

3. Multilevel Inheritance:

- Java supports multilevel inheritance, where a subclass can become a superclass for another subclass, forming a chain of inheritance.

Example of multilevel inheritance:

```
class Animal {  
    // Superclass  
}  
  
class Dog extends Animal {  
    // Subclass  
}  
  
class Labrador extends Dog {  
    // Subclass of Dog (subclass of Animal)  
}
```

Inheritance promotes code reusability, extensibility, and abstraction in object-oriented designs. It allows developers to model real-world relationships and hierarchies effectively, leading to more maintainable and modular code.

Overriding

Method overriding in Java allows a subclass to provide a specific implementation of a method that is already defined in its superclass. This allows for polymorphic behavior, where a subclass can define its own behavior for a method inherited from its superclass. Here are the key points about method overriding in Java:

1. Method Signature:

- To override a method in a subclass, the method must have the same name, parameters, and return type (or covariant return type) as the method in the superclass.

2. `@Override` Annotation:

- While not required, it's a good practice to use the `@Override` annotation when overriding a method. This annotation helps in avoiding accidental

method signature changes that may not actually override a method.

3. Access Modifier:

- The access modifier of the overriding method can be the same or wider than the access modifier of the overridden method in the superclass.
- However, it cannot be narrower. For example, if the superclass method is `public`, the subclass method can be `public` or `protected`, but not `private`.

4. Return Type:

- The return type of the overriding method must be the same as, or a subtype of, the return type of the overridden method. Covariant return types are allowed, which means the subclass method can return a more specific type than the superclass method.

5. Example:

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

6. Dynamic Method Dispatch:

- Method overriding is used in conjunction with dynamic method dispatch, where the actual method to be invoked is determined at runtime based on the type of the object reference.
- This allows for polymorphic behavior, where a reference variable of a superclass type can refer to objects of subclasses and invoke overridden

methods.

Example:

```
Animal myAnimal = new Dog(); // Reference of superclass type,  
object of subclass type  
myAnimal.makeSound(); // Invokes overridden method in Dog cla  
ss
```

Method overriding is a powerful mechanism for achieving polymorphism and code reuse in Java. It allows subclasses to customize behavior inherited from their superclasses, leading to more flexible and modular code.

Abstract Class

In Java, an abstract class is a class that cannot be instantiated directly and may contain abstract methods, concrete methods, or both. Here are the key points about abstract classes in Java:

1. Abstract Class Declaration:

- An abstract class is declared using the `abstract` keyword before the class keyword.
- Abstract classes may contain abstract methods, concrete methods, instance variables, constructors, and static methods.

Example of an abstract class:

```
public abstract class Animal {  
    // Abstract method (no implementation)  
    public abstract void makeSound();  
  
    // Concrete method (with implementation)  
    public void sleep() {  
        System.out.println("Animal is sleeping");  
    }  
}
```

1. Abstract Methods:

- An abstract method is a method declared without an implementation (no method body).
- Abstract methods are intended to be implemented by subclasses.
- Subclasses must provide a concrete implementation for all abstract methods unless the subclass itself is abstract.

2. Concrete Methods:

- Concrete methods are regular methods with an implementation (method body).
- They can be directly used by instances of the abstract class.

3. Usage:

- Abstract classes are useful for defining common behavior and characteristics that subclasses can inherit.
- They provide a way to enforce a contract on subclasses by requiring them to implement certain methods.

4. Subclassing Abstract Classes:

- Subclasses of abstract classes must provide implementations for all abstract methods unless they are also declared as abstract classes.

Example of a subclass extending an abstract class:

```
public class Dog extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

1. Instantiation:

- Abstract classes cannot be instantiated directly with the `new` keyword. Attempting to do so will result in a compilation error.

- However, abstract classes can be used as a reference type, allowing them to refer to instances of their subclasses.

Example:

```
Animal animal = new Dog(); // Valid, using the subclass
```

Abstract classes provide a way to define common behavior and structure for related classes, while allowing subclasses to provide specific implementations for abstract methods. They are an important tool for achieving abstraction, inheritance, and polymorphism in Java.

Final

The `final` keyword is a versatile modifier that can be applied to variables, methods, and classes, each having a specific effect. Here's a detailed explanation of the `final` keyword in each of these contexts:

1. Final Variables

When a variable is declared as `final`, it means that its value cannot be changed once it is initialized. This applies to both primitive data types and reference types.

Example with Primitive Types

```
public class FinalExample {  
    public static void main(String[] args) {  
        final int x = 10;  
        // x = 20; // This will cause a compilation error  
    }  
}
```

Example with Reference Types

```

public class FinalExample {
    public static void main(String[] args) {
        final StringBuilder sb = new StringBuilder("Hello");
        sb.append(" World"); // This is allowed
        // sb = new StringBuilder("Another"); // This will cause a compilation error
    }
}

```

In the case of reference types, the reference itself cannot be changed, but the object it refers to can be modified.

2. Final Methods

A method declared as `final` cannot be overridden by subclasses. This is useful when you want to prevent a method from being altered by any subclass.

Example

```

class Parent {
    public final void show() {
        System.out.println("This is a final method.");
    }
}

class Child extends Parent {
    // public void show() { // This will cause a compilation error
    //     System.out.println("Trying to override.");
    // }
}

```

3. Final Classes

A class declared as `final` cannot be subclassed. This is useful when you want to prevent inheritance for security reasons or to maintain the integrity of the class's

behavior.

Example

```
public final class FinalClass {  
    // Class content  
}  
  
// class SubClass extends FinalClass { // This will cause a c  
ompile error  
//    // Class content  
// }
```

Final Parameters

In addition to variables, the `final` keyword can be used with method parameters. This prevents the parameter from being reassigned within the method.

Example

```
public class FinalParameterExample {  
    public void show(final int x) {  
        // x = 10; // This will cause a compilation error  
        System.out.println(x);  
    }  
}
```

Final with Static

The `final` keyword is often used with `static` to create constants. When combined, `static final` fields are constants that belong to the class rather than any instance, and they cannot be changed after they are initialized.

Example

```
public class Constants {  
    public static final int MAX_USERS = 100;  
    public static final String APP_NAME = "MyApp";  
}
```

Summary of Usage

- **Final Variables:** Prevent reassignment.
- **Final Methods:** Prevent overriding.
- **Final Classes:** Prevent inheritance.
- **Final Parameters:** Prevent reassignment within methods.
- **Static Final Fields:** Create class-level constants.

Using the `final` keyword helps in designing robust and secure applications by controlling how variables, methods, and classes can be modified or extended.

Constructor

A constructor in Java is a special type of method that is called when an object of a class is instantiated. Constructors are used to initialize the state of an object, typically by setting initial values for its fields. They have the same name as the class and do not have a return type, not even `void`.

Key Features of Constructors

1. **Name:** The constructor's name must match the class name exactly.
2. **No Return Type:** Constructors do not have a return type, not even `void`.
3. **Automatically Called:** Constructors are automatically invoked when an object is created using the `new` keyword.
4. **Default Constructor:** If no constructor is explicitly defined, Java provides a default no-argument constructor.

Types of Constructors

1. Default Constructor:

- A no-argument constructor that is automatically provided by Java if no other constructors are defined.
- It initializes the object with default values (e.g., `null` for objects, `0` for numeric types, `false` for boolean).

```
class Example {
    int value;
    String name;
}

public class Test {
    public static void main(String[] args) {
        Example obj = new Example(); // Default constructor is called
        System.out.println(obj.value); // Outputs: 0
        System.out.println(obj.name); // Outputs: null
    }
}
```

2. Parameterized Constructor:

- A constructor that takes arguments to initialize the object's fields with specific values.

```
class Example {
    int value;
    String name;

    Example(int value, String name) {
        this.value = value;
        this.name = name;
    }
}
```

```

public class Test {
    public static void main(String[] args) {
        Example obj = new Example(42, "Java");
        System.out.println(obj.value); // Outputs: 42
        System.out.println(obj.name); // Outputs: Java
    }
}

```

3. No-Argument Constructor:

- A constructor that does not take any arguments but can be explicitly defined to perform specific initializations.

```

class Example {
    int value;
    String name;

    Example() {
        this.value = 10;
        this.name = "Default";
    }
}

public class Test {
    public static void main(String[] args) {
        Example obj = new Example();
        System.out.println(obj.value); // Outputs: 10
        System.out.println(obj.name); // Outputs: Default
    }
}

```

Constructor Overloading

Just like methods, constructors can be overloaded, meaning you can have multiple constructors with different parameter lists within the same class.

```

class Example {
    int value;
    String name;

    // No-argument constructor
    Example() {
        this.value = 0;
        this.name = "No Name";
    }

    // Parameterized constructor
    Example(int value) {
        this.value = value;
        this.name = "No Name";
    }

    // Another parameterized constructor
    Example(int value, String name) {
        this.value = value;
        this.name = name;
    }
}

public class Test {
    public static void main(String[] args) {
        Example obj1 = new Example();
        Example obj2 = new Example(42);
        Example obj3 = new Example(42, "Java");

        System.out.println(obj1.value + ", " + obj1.name); //
Outputs: 0, No Name
        System.out.println(obj2.value + ", " + obj2.name); //
Outputs: 42, No Name
        System.out.println(obj3.value + ", " + obj3.name); //
Outputs: 42, Java
    }
}

```

```
}  
}
```

Polymorphism

Polymorphism in Java is a fundamental concept in object-oriented programming that allows objects to be treated as instances of their parent class rather than their actual class. This promotes flexibility and integration by enabling a single interface to represent different underlying forms (data types). Java achieves polymorphism in two primary ways: compile-time (or static) polymorphism and runtime (or dynamic) polymorphism.

Types of Polymorphism

1. **Compile-time Polymorphism (Static Polymorphism)**
2. **Runtime Polymorphism (Dynamic Polymorphism)**

Compile-time Polymorphism

Compile-time polymorphism is achieved through method overloading and operator overloading.

Method Overloading

Method overloading occurs when multiple methods in the same class have the same name but different parameters (different type, number, or both).

```
class Example {  
    void display(int a) {  
        System.out.println("Argument: " + a);  
    }  
    void display(String b) {  
        System.out.println("Argument: " + b);  
    }  
}
```

```

public class Test {
    public static void main(String[] args) {
        Example obj = new Example();
        obj.display(10); // Calls the method with int parameter
        obj.display("Hello"); // Calls the method with String parameter
    }
}

```

In this example, the `display` method is overloaded to handle different types of arguments.

Runtime Polymorphism

Runtime polymorphism is achieved through method overriding. It occurs when a subclass provides a specific implementation of a method that is already defined in its superclass.

Method Overriding

Method overriding allows a subclass to provide a specific implementation for a method that is already defined in its superclass.

```

class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

```

```

public class Test {
    public static void main(String[] args) {
        Animal obj = new Dog();
        obj.sound(); // Calls the overridden method in Dog c
lass
    }
}

```

In this example, the `sound` method is overridden in the `Dog` class, and the actual method to be called is determined at runtime based on the object type.

Benefits of Polymorphism

1. **Code Reusability:** Polymorphism promotes the reuse of code by allowing the same interface to be used for different implementations.
2. **Flexibility and Maintainability:** Code is more flexible and easier to maintain because methods can be added, modified, or overridden without affecting existing functionality.
3. **Extensibility:** New subclasses can be added with specific implementations without altering existing code.

Key Concepts

- **Inheritance:** Polymorphism relies on inheritance, where a subclass inherits properties and methods from a superclass.
- **Method Overriding:** A subclass can provide a specific implementation for a method already defined in its superclass.
- **Method Overloading:** Multiple methods with the same name but different parameters within the same class.

Example of Polymorphism

Here is an example demonstrating both compile-time and runtime polymorphism:

```

class Shape {
    void draw() {

```

```

        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing a rectangle");
    }
}

public class PolymorphismExample {
    public static void main(String[] args) {
        // Compile-time polymorphism
        display(5);
        display("Java");

        // Runtime polymorphism
        Shape shape;
        shape = new Circle();
        shape.draw(); // Outputs "Drawing a circle"

        shape = new Rectangle();
        shape.draw(); // Outputs "Drawing a rectangle"
    }

    // Overloaded methods
    static void display(int a) {
        System.out.println("Argument: " + a);
    }
    static void display(String b) {

```

```
        System.out.println("Argument: " + b);
    }
}
```

Parent & Child

Parent Object (Superclass Instance)

A parent object is an instance of a superclass. A superclass is a class that is extended by another class (the subclass). The superclass contains common attributes and behaviors that are shared by all subclasses.

Child Object (Subclass Instance)

A child object is an instance of a subclass. A subclass is a class that inherits from a superclass. It can inherit and override attributes and behaviors from the superclass, and it can also have additional attributes and behaviors of its own.

Example Scenario

Let's define a simple inheritance hierarchy to illustrate the concepts of parent and child objects.

```
class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    void makeSound() {
        System.out.println(name + " makes a sound");
    }
}

class Dog extends Animal {
    Dog(String name) {
```

```

        super(name);
    }

    @Override
    void makeSound() {
        System.out.println(name + " barks");
    }

    void fetch() {
        System.out.println(name + " fetches a ball");
    }
}

class Cat extends Animal {
    Cat(String name) {
        super(name);
    }

    @Override
    void makeSound() {
        System.out.println(name + " meows");
    }

    void scratch() {
        System.out.println(name + " scratches");
    }
}

```

Creating Parent and Child Objects

Parent Object

```

public class TestParentObject {
    public static void main(String[] args) {
        Animal animal = new Animal("Generic Animal");
    }
}

```

```

        animal.makeSound(); // Outputs "Generic Animal makes
a sound"
    }
}

```

Child Objects

```

public class TestChildObjects {
    public static void main(String[] args) {
        Dog dog = new Dog("Buddy");
        dog.makeSound(); // Outputs "Buddy barks"
        dog.fetch();      // Outputs "Buddy fetches a ball"

        Cat cat = new Cat("Whiskers");
        cat.makeSound(); // Outputs "Whiskers meows"
        cat.scratch();   // Outputs "Whiskers scratches"
    }
}

```

Explanation

1. Parent Object:

- The `animal` variable is an instance of the `Animal` class (the parent class). It has access to the `makeSound` method defined in the `Animal` class.
- `Animal animal = new Animal("Generic Animal");` creates a new `Animal` object with the name "Generic Animal".

2. Child Objects:

- The `dog` variable is an instance of the `Dog` class (a child class of `Animal`). It has access to the `makeSound` method from `Animal`, which it overrides, and it also has its own method `fetch`.
- `Dog dog = new Dog("Buddy");` creates a new `Dog` object with the name "Buddy".
- The `cat` variable is an instance of the `Cat` class (another child class of `Animal`). It overrides the `makeSound` method and has its own method `scratch`.

- `Cat cat = new Cat("Whiskers");` creates a new `Cat` object with the name "Whiskers".

Upcasting and Downcasting

When dealing with parent and child objects, upcasting and downcasting are common operations.

Upcasting

Upcasting is the process of treating a child object as if it were a parent object. This is done implicitly and allows you to use a child object where a parent object is expected.

```
Animal animal = new Dog("Buddy");  
animal.makeSound(); // Outputs "Buddy barks"
```

In this example, `Dog` is upcast to `Animal`. The `animal` reference can only call methods available in the `Animal` class, even though it points to a `Dog` object.

Downcasting

Downcasting is the process of treating a parent object as if it were a child object. This requires an explicit cast and should be done carefully, often with a type check.

```
Animal animal = new Dog("Buddy");  
Dog dog = (Dog) animal; // Downcast Animal to Dog  
dog.fetch(); // Outputs "Buddy fetches a ball"
```

Real-World Analogy

Consider a general vehicle and specific types of vehicles:

- **Vehicle (Parent Class):** A general concept of a vehicle, with common properties like `speed` and behaviors like `start`.
- **Car and Bicycle (Child Classes):** Specific types of vehicles with additional properties and behaviors.

```

class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}

class Car extends Vehicle {
    void start() {
        System.out.println("Car starts");
    }

    void honk() {
        System.out.println("Car honks");
    }
}

class Bicycle extends Vehicle {
    void start() {
        System.out.println("Bicycle starts");
    }

    void ringBell() {
        System.out.println("Bicycle rings bell");
    }
}

```

Using these classes:

```

public class TestVehicles {
    public static void main(String[] args) {
        Vehicle vehicle;

        vehicle = new Car();
        vehicle.start(); // Outputs "Car starts"
    }
}

```

```

        vehicle = new Bicycle();
        vehicle.start(); // Outputs "Bicycle starts"
    }
}

```

If you need to access subclass-specific methods, you would downcast:

```

public class TestDowncasting {
    public static void main(String[] args) {
        Vehicle vehicle = new Car();
        if (vehicle instanceof Car) {
            Car car = (Car) vehicle;
            car.honk(); // Outputs "Car honks"
        }
    }
}

```

Summary

- **Parent Object:** An instance of the superclass.
- **Child Object:** An instance of the subclass that inherits from the superclass.
- **Upcasting:** Implicitly treating a child object as a parent object.
- **Downcasting:** Explicitly treating a parent object as a child object, usually with type checking to avoid `ClassCastException`.

Understanding these concepts helps in designing flexible and reusable object-oriented systems.

Incorrect Downcasting

When you try to downcast an object to a subclass that it does not actually belong to, a `ClassCastException` is thrown.

Example:

```

class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Cat meows");
    }
}

public class TestClassCastException {
    public static void main(String[] args) {
        Animal animal = new Dog();

        // Incorrect downcasting, animal is not a Cat
        Cat cat = (Cat) animal; // This will throw ClassCast
Exception at runtime
        cat.makeSound();
    }
}

```

Type Checking to Prevent **ClassCastException**

In Java, type checking is essential when downcasting to ensure that the object being cast is actually of the target type. This is crucial because downcasting

incorrectly can lead to a `ClassCastException`, which occurs when you try to cast an object to a class of which it is not an instance.

What is `ClassCastException` ?

`ClassCastException` is a runtime exception that is thrown when an attempt is made to cast an object to a subclass of which it is not an instance. This exception can be avoided by using type checking before performing a downcast.

Using `instanceof` for Type Checking

The `instanceof` operator is used to check whether an object is an instance of a specific class or subclass. This operator returns `true` if the object is an instance of the specified type, and `false` otherwise.

Example Scenario

Let's revisit the `Animal`, `Dog`, and `Cat` classes:

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }

    void fetch() {
        System.out.println("Dog fetches a ball");
    }
}

class Cat extends Animal {
    @Override
```

```

void makeSound() {
    System.out.println("Cat meows");
}

void scratch() {
    System.out.println("Cat scratches");
}
}

```

Downcasting with Type Checking

Here's how you can use `instanceof` to safely downcast:

```

public class DowncastingExample {
    public static void main(String[] args) {
        Animal animal = new Dog(); // Upcast Dog to Animal

        if (animal instanceof Dog) {
            Dog dog = (Dog) animal; // Downcast Animal to Do
g
            dog.makeSound(); // Outputs "Dog barks"
            dog.fetch(); // Outputs "Dog fetches a ball"
        } else {
            System.out.println("The object is not a Dog.");
        }

        animal = new Cat(); // Upcast Cat to Animal

        if (animal instanceof Cat) {
            Cat cat = (Cat) animal; // Downcast Animal to Ca
t
            cat.makeSound(); // Outputs "Cat meows"
            cat.scratch(); // Outputs "Cat scratches"
        } else {
            System.out.println("The object is not a Cat.");
        }
    }
}

```

```
}  
}
```

Explanation

1. Upcasting:

- `Animal animal = new Dog();`
 - A `Dog` object is created and implicitly upcast to `Animal`. The `animal` reference now points to a `Dog` object.

2. Type Checking Using `instanceof`:

- `if (animal instanceof Dog)`
 - This condition checks if the `animal` reference actually points to an instance of `Dog`.
 - If true, the object can safely be downcast to `Dog`.

3. Downcasting:

- `Dog dog = (Dog) animal;`
 - Perform the downcast. Since the type check passed, this cast is safe, and the `dog` reference can now access methods specific to the `Dog` class.

4. Accessing Subclass Methods:

- `dog.makeSound();` and `dog.fetch();`
 - Call methods specific to the `Dog` class.

5. Reassigning and Type Checking for Another Subclass:

- `animal = new Cat();`
 - The `animal` reference now points to a `Cat` object.
- `if (animal instanceof Cat)`
 - Check if `animal` is an instance of `Cat`.
- `Cat cat = (Cat) animal;`

- Safely downcast to `Cat` and access `Cat` specific methods.

Deadly Diamond of Death

The "Deadly Diamond of Death," often simply referred to as the "Diamond Problem," is a well-known issue in multiple inheritance in object-oriented programming. It occurs when a class inherits from two classes that both derive from a common base class. This can lead to ambiguity and complications in the inheritance structure. The problem is named for the diamond-shaped inheritance diagram that this situation creates.

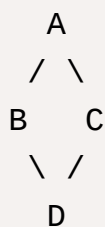
Here's a detailed explanation of the Diamond Problem:

Structure

Imagine the following classes:

- **Class A:** The base class.
- **Class B** and **Class C:** Both inherit from **Class A**.
- **Class D:** Inherits from both **Class B** and **Class C**.

The inheritance structure forms a diamond shape:



Issues

When **Class D** inherits from both **Class B** and **Class C**, it inherits the members (attributes and methods) of **Class A** twice: once through **Class B** and once through **Class C**. This creates several issues:

1. **Ambiguity:** If **Class A** has a method (e.g., `methodA`), and **Class D** calls `methodA`, it is ambiguous which version of `methodA` should be called: the one inherited via **Class B** or the one inherited via **Class C**.
2. **Redundancy:** The attributes and methods of **Class A** are duplicated in **Class D**, which is inefficient and can lead to inconsistent states if both copies are not kept in sync.

But Java avoids the Diamond Problem altogether by not supporting multiple inheritance of classes. Instead, it allows multiple inheritance through interfaces. Since interfaces do not hold state (attributes), the ambiguity problem is avoided.

Interface

In Java, interfaces are a fundamental part of its object-oriented programming paradigm, providing a way to achieve abstraction and multiple inheritance. An interface in Java is a reference type, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types. The methods in interfaces are abstract by default, meaning they do not have a body and must be implemented by classes that choose to implement the interface.

Here are the key aspects and features of interfaces in Java:

Key Features of Interfaces

1. **Abstract Methods:** An interface primarily contains abstract methods. These are method signatures without a body. Any class that implements the interface must provide concrete implementations for these methods.

```
public interface Animal {  
    void eat();  
    void sleep();  
}
```

2. **Constants:** An interface can contain constants, which are implicitly `public`, `static`, and `final`.

```
public interface Animal {  
    int LEGS = 4;  
}
```

3. **Default Methods:** Java 8 introduced default methods, which are methods in an interface that have a body. This allows interfaces to have some method implementations.

```
public interface Animal {  
    default void breathe() {  
        System.out.println("Breathing...");  
    }  
}
```

4. **Static Methods:** Interfaces can also have static methods. These methods belong to the interface class and can be called without an instance of the implementing class.

```
public interface Animal {  
    static void commonBehavior() {  
        System.out.println("Animals share common behavior  
s.");  
    }  
}
```

5. **Multiple Inheritance:** Through interfaces, Java allows a form of multiple inheritance. A class can implement multiple interfaces, thereby inheriting the abstract methods of all interfaces.

```
public interface Animal {  
    void eat();  
}
```

```

public interface Pet {
    void play();
}

public class Dog implements Animal, Pet {
    public void eat() {
        System.out.println("Dog is eating.");
    }

    public void play() {
        System.out.println("Dog is playing.");
    }
}

```

Implementing Interfaces

When a class implements an interface, it must provide concrete implementations for all the abstract methods declared in the interface. If it does not, the class must be declared abstract.

```

public interface Animal {
    void eat();
    void sleep();
}

public class Dog implements Animal {
    @Override
    public void eat() {
        System.out.println("Dog is eating.");
    }

    @Override
    public void sleep() {
        System.out.println("Dog is sleeping.");
    }
}

```

```
}  
}
```

Interface Inheritance

Interfaces can extend other interfaces, just like classes. This allows for the creation of more specific interfaces based on more general ones.

```
public interface Animal {  
    void eat();  
}  
  
public interface Pet extends Animal {  
    void play();  
}  
  
public class Dog implements Pet {  
    @Override  
    public void eat() {  
        System.out.println("Dog is eating.");  
    }  
  
    @Override  
    public void play() {  
        System.out.println("Dog is playing.");  
    }  
}
```

Practical Use Cases

1. **Decoupling Code:** Interfaces are used to define contracts that separate what a class can do from how it does it. This decouples the implementation details from the usage.
2. **Multiple Inheritance:** Since Java does not support multiple inheritance of classes, interfaces provide a way to achieve similar functionality.

3. **Plug-and-Play Architecture:** Interfaces allow for the creation of interchangeable components. For example, different classes can implement the same interface in different ways, allowing them to be used interchangeably in the application.

Example

Here's a complete example demonstrating the use of interfaces:

```
public interface Animal {
    void eat();
    void sleep();

    default void breathe() {
        System.out.println("Breathing...");
    }

    static void commonBehavior() {
        System.out.println("All animals exhibit some common behaviors.");
    }
}

public interface Pet {
    void play();
}

public class Dog implements Animal, Pet {
    @Override
    public void eat() {
        System.out.println("Dog is eating.");
    }

    @Override
    public void sleep() {
        System.out.println("Dog is sleeping.");
    }
}
```

```

    }

    @Override
    public void play() {
        System.out.println("Dog is playing.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat();
        dog.sleep();
        dog.play();
        dog.breathe();

        // Call the static method in the interface
        Animal.commonBehavior();
    }
}

```

In this example, the `Dog` class implements both `Animal` and `Pet` interfaces, providing concrete implementations for all abstract methods and using the default method from `Animal`. This demonstrates the flexibility and power of using interfaces in Java.